

FLASH Development Guide

Last updated February 10, 2026.

Written by Daniel Terry and Manuel Juette.

1 TABLE OF CONTENTS

2	Introduction	2
3	Prerequisites	3
4	Software Architecture	4
4.1	Subsystems	4
4.2	Class Hierarchy	5
4.3	File Structure	7
4.4	Configuration Files	8
4.5	Automation	8
5	Execution Walkthrough	9
5.1	Application Startup	9
5.2	Application Shutdown	9
5.3	Error Handling	10
5.4	Show Live / Stop Live	10
5.5	Stream Acquisition	11
6	Adding support for new devices	13
6.1	Modifying an Existing Device Actor	13
6.2	Creating a New <i>Device</i> Actor	14
7	Compiling an executable	15
8	C++ libraries	16
8.1	Conversion to TIFF (dcimage2tiff)	16
8.2	Particle Counter	16
9	Team Development and Other Tips	17
10	Troubleshooting	18
10.1	Error Codes	18
10.2	Log Files	19
11	Style Guide	20

2 INTRODUCTION

FLASH is a microscope instrument control and data acquisition software tool specialized for widefield single-molecule fluorescence imaging, especially single-molecule FRET, written to be highly modular and parallelized to support new devices, workflows, and user interfaces relatively easily. Read “FLASH Documentation.pdf” to review the normal functioning of the program.

The purpose of this document is to provide an overview of the software architecture, implementation strategies, team development tips, and style guidelines. Because FLASH is written primarily in LabView, it does not have the extensive inline documentation of text-based languages, making this document especially important for understanding how FLASH works, how to add features, and how to fix bugs. If you are new to LabView, we suggest first reviewing the [LabVIEW programming paradigms](#) and the various guides for the [Actor Framework](#).

The image below provides a high-level overview of the modules in FLASH and how they are connected. Files are orange, software modules in green, and physical hardware devices in blue.

After installing all prerequisites listed in the next section, deploy the FLASH source code to “C:\FLASH\Development”. To run FLASH, open the project file, double click on “FLASH.vi” from within the LabVIEW project, and click the “play” button to start the program. Always open the project file (.lvproj) in the FLASH directory to examine the code or make any changes. **Avoid running or modifying files outside the project, which will cause link errors.**

3 PREREQUISITES

The following programs and libraries must be installed before loading the FLASH source code to avoid link errors. The associated devices are not required to be installed, but at least one physical camera must be connected to run the program (there is no simulated camera functionality). If necessary, you can avoid some driver dependencies by removing the device .lvlib from the project file, assume the device actor class is always launched by path string, which is almost always true.

- **64-bit LabVIEW Professional version 2023 Q3.** This requires a license to be purchased.
<https://www.ni.com/en-us/shop/labview.html>
 - JSONtext from JDP Science (download using VI Package Manager).
- **Hamamatsu** Video Capture library for LabVIEW. This will install VIs into LabVIEW user.lib folder and install tmcamcon.dll to C:\Windows\System32.
<https://dcam-api.com/hamamatsu-software/>
- **Andor** SDK2 (for EMCCD cameras): tested with version 2.104.30084.0.
<https://andor.oxinst.com/downloads/uploads/Software%20Development%20Kit.pdf>
<https://andor.oxinst.com/downloads/view/andor-sdk-2.104.30084.0>
- **Photometrics** PVCAM and LabVIEW adapter.
<https://www.photometrics.com/products/pvcam>
<https://www.photometrics.com/support/third-party-software/labview>
 - NOTE: the LabView adapter by default installs in the “Public Documents” folder, which causes linking issues in LabView. Instead, we recommend moving this folder into user.lib. The expected path is: “C:\Program Files\National Instruments\LabVIEW 2023\user.lib\TPM LabVIEW adapter\”.
- **Nikon Ti2** SDK 64-bit, which includes the device driver for the Ti2 microscope.
<https://nisdsk.recollective.com/microscopes>
- **Thorlabs Kinesis** SDK for LabVIEW.
https://www.thorlabs.com/software_pages/ViewSoftwarePage.cfm?Code=Motion_Control
- **ASI stage controller** driver for LabVIEW, which installs in user.lib.
<https://www.asiimaging.com/support/downloads/ms-2000-control-using-nis-labview-and-the-serial-port/>
- **Coherent Genesis:** You can install the “OPSL GUI”, which includes both the driver and test UI, or install the device driver and a UI for testing, or “CDM v2.12.28 WHQL Certified” if you just want the driver.
<https://www.coherent.com/resources?query=opsl%20gui>
<https://www.coherent.com/resources?query=CDM%20v2.12.28%20WHQL%20Certified>
- **Visual Studio** with C/C++ tool chain (free Community Edition is sufficient) to recompile the C++ DLLs (specifically dcimg2tiff.dll) used for time-critical operations in FLASH.
- Optional: instructions for installing software for configuring the Arduino are listed on the developer’s website: <https://github.com/stjude-smc/microsync>

4 SOFTWARE ARCHITECTURE

4.1 SUBSYSTEMS

FLASH is divided into five subsystems with distinct roles:

1. **User Interface (UI) layer.** These actors handle all user interactions, communicating this information as messages to the *Application* actor. There is never direct interaction between UI and hardware actors. The UI does not maintain state – it simply mirrors the state of the *Application*, as indicated by status messages. The UI checks the validity of input parameters only for user convenience; the *Application* actor performs any critical parameter checks.
 - a. *Main Window*: primary user interface for FLASH and provides most program control.
 - b. *Viewer*: displays frame data, streaming status, and live analysis (e.g., particle counts).
2. **Application layer.** This layer includes only one actor: *Application*. It is the core system that loads the instrument configuration, controls the lifecycle of all the other Actors, and translates high-level commands from the UI into a specific sequence of device commands (HAL methods), after verifying input parameters. The *Application* maintains the state of the program.
 - a. *AutomationContext* is a class within the application layer that provides methods for translating a *Protocol* (parameters from the UI that describe a movie series) into a run plan (array of actions and associated parameters to be sent to the *Application* actor in sequence). This class keeps track of the current step in the run, freeing up the *Application* actor to only consider the immediate step being executed.
3. **Hardware abstraction layer (HAL).** Classes in this layer provides abstract interfaces for mapping general device actions (like “start streaming”) into device-specific commands. Each class in this layer encapsulates the functionality of a device type (*Camera*, *Laser*, etc.). The public methods of the class are called by the *Application* actor via a message, describing a high-level command. The public method then calls protected methods in the same class that abstract device-specific commands (such as “prepare camera”). These protected methods (“hooks”) are overridden by device-specific classes so that dynamic dispatch can be used to call the correct device-specific class. HAL classes do not maintain state.
4. **Device layer.** Actors in this layer all inherit from a specific HAL class, overriding methods to translate a high-level device action into device-specific code (such as DLL calls). These actors independently monitor the state of device and report this information, including errors, as messages back to the *Application* actor. When several physical devices are represented by a single driver/API (such as cameras), these should be supported by a single underlying actor to avoid race conditions.
5. **Services.** These classes asynchronously execute computationally intensive tasks, such as saving frame data to disk and live image analysis, the results of which are not immediately needed by the application.

4.2 CLASS HIERARCHY

The main classes within FLASH are Actors, which are derived from the *Actor* class of the Actor Framework built into LabVIEW. The core *Application* actor implements the Application subsystem. *UI Window* and *Device* are abstract classes that provide generic functionality for the UI and Hardware subsystems; specific windows and devices are implemented by subclassing. Messages between actors are derived from a separate *Message* class, which is also part of the framework.

Actor (LabVIEW built in type)

- Application
- TIFF Writer (Service)
- Particle Counter (Service)
- Timer (Service)
- UI Window
 - Main Window
 - Viewer
 - Splash Screen
- Device
 - Autosampler
 - 1. Altesta Autosampler (SIELC)
 - 2. Simulated Autosampler
 - Camera
 - 1. Hamamatsu
 - 2. Andor (SDK2)
 - 3. Photometrics
 - Laser
 - 1. LaserQuantum
 - 2. Coherent (Obis and HOPS families)
 - 3. Thorlabs (polarizer motion control)
 - 4. Cobolt
 - Microscope Stand
 - 1. Nikon Ti2
 - Power Meter
 - 1. NewPort
 - 2. Thorlabs
 - Stage Controller
 - 1. Nikon Ti2
 - 2. ESP30x
 - 3. Ludl
 - 4. ASI
 - 5. Simulated Stage
 - Sync Device
 - 1. NI DAQ
 - 2. Arduino DAQ ([Mega2560](#); legacy support)
 - 3. Microsync

AutomationContext
Protocol
TIFF

4.3 FILE STRUCTURE

Below is the virtual file structure of the project, which may not perfectly match the file structure on disk, but is very similar:

- **Application:** Application actor, AutomationContext, Protocol classes
- **BinaryTIFF:** TIFF class, Tiff Writer actor, and dcm2tiff (C++ project)
- **Hardware**
 - **Hardware Abstraction Layer:** abstract classes defining interfaces for each device type
 - **Hardware Devices:** concrete child classes implementing support for specific devices
- **Protocols:** .json files describing the sequence of steps for executing parameters sweeps and other automated workflows.
- **Resources**
 - **Config:** JSON files defining instrument configurations.
 - **Dependencies:** external libraries used by FLASH.
 - **dll:** binary dependencies
 - **GLOBAL:** global variables (to be removed in a future version)
 1. *GLOBAL Experiment Metadata.vi:* used for collecting device settings in real time to be used when saving the metadata for output TIFF files.
 2. *GLOBAL Time-Critical.vi:* used for data sharing of a small number of variables such as frame data to reduce message-passing overhead.
 - **Helper VIs:** stand-alone functions that are used throughout the codebase. Any VI's that have a functional scope within one module should be moved into that lplib instead.
 - **Images:** artwork and icons used for compiling the application.
 - **Testing:** VI's used to isolate specific actors or modules for testing.
 - **docs:** documentation, user guide, dev guide, release notes, etc.
- **Services** (Particle Counter, Timer actors)
- **User Interface** (Main Window, Viewer, Splash Screen actors)
 - **Dialogs:** modal dialog VI's used by the Main Window actor.

4.4 CONFIGURATION FILES

FLASH stores instrument configuration settings as JSON files in the “Config” folder. The JSON format is compact, human-readable text that is robust to changes in the type definitions between versions. For example, when data is not available in the config file, the default value is used. Extraneous values/fields are silently ignored. This allows old configuration files to be used by new versions of FLASH even when the *Hardware Configuration* typedef has changed significantly.

The format of the file is just a serialized version of the *Hardware Configuration.vi* typedef cluster, which includes a cluster for each device type. The *Create Hardware Configuration UI.vi* dialog provides a visual representation of this typedef cluster. Enums are matched by their text value, so they are robust to expansions and reordering of enum values, but not to changes in the text names.

The *Init.json* file in the main project directory contains general settings, including a map of computer host names and default configuration files to use for each. This mechanism aids in deploying the software to facilities with a large number of microscopes. This file can be edited with a text editor or you can use the *Create Initialization File.vi* dialog.

4.5 AUTOMATION

FLASH is designed for workflow automation. The *Application* actor focuses only on executing one step at a time, keeping its state management simple. The AutomationContext object, which is stored in *Application* private data, executes a sequence of commands that make up the desired workflow.

Below is a list of the relevant objects and typedefs that coordinate this process:

- **Protocol:** encapsulates the high-level parameters describing a movie sequence and includes a method to check the validity of these the settings against the current hardware configuration.
- **AutomationContext:** encapsulates methods to build (compile) and execute the Run Plan.
- **Run Plan:** array of Parallel Blocks describing the full sequence of planned events.
- **Parallel Block:** array of Tasks that will be executed simultaneously.
- **Task:** cluster of Action, Parameters (variant), and correlation ID (integer).
- **Action:** enum that can be directly mapped to *Application*’s public methods, such as next field, autofocus, set laser power, start streaming, etc.
- **Correlation ID:** unique value used to match notifications of a completed device task against those pending in the Run Plan. The values are sequential for the lifetime of the program.

The normal program flow has *Main Window* creating a Protocol object that is translated into a Run Plan for execution, but these classes could be extended to provide more complex utilities for creating a Run Plan or loading one from file.

5 EXECUTION WALKTHROUGH

This section provides an overview of the program flow, focusing on the key methods.

5.1 APPLICATION STARTUP

Below are the steps executed on successful startup:

1. **FLASH.vi** launches the *Application* as root actor.
2. **Application/Startup.vi** launches the *Splash Screen Actor*.
3. **Application/Load Configuration.vi** dialog allows user to select the hardware configuration, saving this information in the *Application* private data. The “Create Hardware Configuration UI.vi” dialog is used for modifying an existing configuration or creating a new one.
4. **Application/Init HW.vi** initialize configured hardware:
 - a. **Application/Initialize Lasers.vi**: launches *Laser* actors (up to 4).
 - b. **Application/Initialize Virtual Microscope.vi**: launches *Microscope* actor.
 - c. **Application/Initialize Stage Controller.vi**: launches *Stage Controller* actor.
 - d. **Application/Init Power Meter.vi**: launches *Power Meter* actor.
 - e. **Application/Initialize Shutters.vi**: launches *Sync Device* actor.
 - f. **Application/Initialize Virtual Camera.vi**: launches *Camera* actor.
 - g. **Application/Initialize UI.vi**: launches service actors and initializes *AutomationContext*.
 - h. **Application/Init Telemetry.vi**: sends telemetry data to server for usage statistics.
5. **Application/Startup.vi** launches the *Main Window* actor and closes the splash screen.

“Initialize <device>.vi” methods launch child actors by class path so that dependencies are only loaded as needed, which may not all be present in production systems. The device’s *Initialize.vi* method is called directly using the object’s data wire, rather than by message, to ensure the startup process is sequential and deterministic to avoid race conditions.

The device initialization order is most arbitrary, except that (1) *Sync Device* provides a jitter parameter to *Camera* for timing calculations and (2) *Lasers* are initialized first to ensure the laser power is set to zero before *Sync Device* for extra safety.

5.2 APPLICATION SHUTDOWN

Below is the execution flow for normal (user-initiated) shutdown:

1. *Main Window* sends *Shut Down Msg* to *Application*.
2. **Application/Cancel Current Operation.vi** terminates running operations (such as streaming).

3. ***Application/Shut Down.vi*** sends *Stop Msg* to all running actors and reopens the splash screen.
4. ***<Device>/Shut Down.vi*** puts each physical device in a safe state (such as escaping the objective and closing shutters) and frees allocated resources like file handles.
5. ***Application/Handle Last Ack Core.vi*** decrements a counter when each child actor is terminated and, when the counter reaches zero, sends *Stop Msg* to *Application*.
6. ***Application/Stop Core.vi*** closes the log file and the splash screen.

This shut down procedure ensures that the log file is properly closed only after all actors have terminated, which helps with troubleshooting actors that freeze on shutdown. It also ensures the splash screen only closes when the application has actually terminated.

5.3 ERROR HANDLING

Below is a description of how error handling should be done in FLASH; in practice some legacy code does not follow this pattern.

- **Minor errors** are handled entirely within the device method. “Minor” means that the current acquisition process can continue without interruption. An example would be a bad power meter reading, where the result isn't important to the acquisition. Another example is a blocking command that can be repeated quickly and safely.
- **Serious but non-fatal errors** are caught in device methods, logged in detail, converted to a relevant program-level error code, and passed to Application using the "cancel current operation" method, including the error code. An example would be an invalid or unsafe command sent to an autosampler -- the device is in unstable state and we need to reset before continuing. The Application will convert the error code into a plain-language warning dialog (using an int->string map/registry) with suggested next steps to recover.
- **Fatal or unknown errors** propagate to the output terminal of an actor method, where they are handled in Actor/Handle Error.vi. The device actor will be terminated and the Application will receive a Last Ack message with the final error cluster, which will be displayed and the user offered a choice whether to continue. If the user says yes, the next command sent to the device will still crash the application (because the reference is now invalid).
- **"Handle Error.vi" overrides** can provide an additional safety net to clean up and put the device in a safe state after a fatal error but should still propagate that error to the application and terminate the actor. Don't use this override for normal error handling.
- Initialization methods are a special case because they are called directly by the Application. But in practice, they are handled the same way, except that errors propagating through the method end up directly on the Application's error wire and prevent the Application from starting.

5.4 SHOW LIVE / STOP LIVE

This section describes events after user clicks the “Show Live” button in *Main Window*:

- 1) *Main Window* sends *Start Live Msg* to *Application* with current acquisition parameters.
- 2) ***Application/Start Live.vi*** validates settings, sends *Start Live Mode Msg* to *Camera*, *Start Streaming Msg* to *Sync Device*, *Application Mode Change Msg* to *Main Window*, and *Set Visibility Msg* to *Live Viewer*.
- 3) ***<Camera>/Start Live Mode.vi*** configures camera(s) for frame acquisition.
- 4) ***<Sync Device>/Start Streaming.vi*** starts TTL pulse sequence to other devices.
- 5) ***<Camera>/Get Most Recent Frame.vi*** saves the most recently acquired frame and index to the “Live Frame” and “Current Frame” global variables, respectively, and sends *New Frame Notify Msg* to *Application*.
- 6) ***Application/New Frame Notify.vi*** sends *New Frame Msg* to the *Particle Counter* service actor.
- 7) ***Viewer/Actor Core.vi*** renders the most recent frame from “Live Frame” global variable.
- 8) Steps 7-9 repeat until the user closes the *Viewer* window or clicks “Stop Live” in *Main Window*, both of which send “Stop Live Msg” to *Application*.
- 9) ***Application/Stop Live Mode.vi*** sends *Stop Live Msg* to *Camera*, *Stop Streaming* to *Sync Device*, *Application Mode Change Msg* to *Main Window*, and *Set Visibility Msg* to *Viewer*.

5.5 STREAM ACQUISITION

This program sequence occurs when the user clicks “Start Streaming” in the *Main Window* front panel. See the section above with a more complete description of *AutomationContext* and *Protocol* objects.

- 1) A *Start Recording Msg* with a newly created *Protocol* object is sent to *Application*.
- 2) ***Application/Start Recording.vi*** checks *Protocol* validity against the hardware configuration.
- 3) ***AutomationContext/Compile Protocol.vi*** and builds a Run Plan, which will include the full sequence of steps for the acquisition, including stage movement, setting laser power, Z stack acquisition for autofocus, and streaming, along with parameter values for each. For brevity, we consider the simplest case below.
- 4) ***AutomationContext/Next Step.vi*** pulls all Tasks in the next Parallel Block in the Run Plan and sends them as messages to *Application* with their associated correlation ID. In the simplest case, this is always *Start Stream Msg*.
- 5) ***Application/Start Stream.vi*** sends *Start Stream Msg* to *Camera*, runs the timer dialog (if applicable), sends *Start Streaming Msg* to *Sync Device*, and opens the *Viewer* window.
- 6) ***<Camera>/Start Stream.vi*** configures camera for frame acquisition and streaming to disk.
- 7) ***<Sync Device>/Start Streaming.vi*** starts TTL pulse sequence to other devices.
- 8) ***<Camera>/Get Most Recent Frame.vi*** saves the most recently acquired frame to the “Live Frame” global variables and sends *New Frame Notify Msg* to *Application*.

- 9) **Application/New Frame Notify.vi** sends *New Frame Msg* to the *Particle Counter* service actor. When the last frame is received, *Finish Stream.vi* is called.
- 10) **Application/Finish Stream.vi** sends *Stop Streaming Msg* to *Sync Device* and *Finish Stream Msg* to *Camera* to end the current acquisition.
- 11) **<Sync Device>/Stop Streaming.vi** ends pulse sequence and resets to idle state.
- 12) **Camera/Finish Stream.vi** stops the acquisition, compiles metadata for final movie file, and sends *Save Tiff Msg* to *TiffWriter*, and sends *Automation_TaskCompleted Msg* to *Application*.
- 13) **Tiff Writer/Save Tiff.vi** prepares a TIFF file with header, *dcimg2tiff.dll* transfers the raw frame data on disk into the TIFF file, and polls communicates the progress to the *Main Window* via the "Saving Frame" global variable. This process runs in the background.
- 14) **Application/Automation_TaskCompleted.vi** executes the next step in the Run Plan, or if all steps are complete, sends *Finish Recording Msg* to *Application* to return to the idle state.

6 ADDING SUPPORT FOR NEW DEVICES

This section provides tips for how to modify FLASH to add support for new devices.

6.1 MODIFYING AN EXISTING DEVICE ACTOR

If FLASH already supports devices from the vendor of the new device, it may be possible to add support for the new device relatively easily. For example, all Hamamatsu cameras use the same DCAM driver and should require relatively small changes to add support. For such cases, below is a brief guide of how each *Device* class would be updated to support a new device type.

Cameras:

- **Hamamatsu:** modify the file *Max Lines Auto.vi*, specifically adding a new case to the “Camera Model” case structure. This is mainly to give plain-text names to the readout modes and specify a triggering overhead term that is not otherwise considered by the driver. You may need to modify *Prepare Acquisition.vi* if the triggering modes have changed.
- **Photometrics:** this may work for new sCMOS cameras without modification because the code is not model-specific. If there are issues with triggering, first check the timing calculations in *PmSetRegion.vi*. The default readout mode (“Auto”) is set in *PmSetReadoutMode.vi*. *Prepare Acquisition.vi* may also need to be modified, particularly if the triggering modes are different than the Kinetix.
- **Andor:** the current code, which is unstable, only support SDK2 for EMCCD cameras. If you want to use a modern sCMOS camera, you will need to create a new *Camera* class for it.

Power meter:

- **Thorlabs:** any power meter using the TLPM library should be supported without modification.

Lasers:

- **Coherent:** all lasers in the OBIS series and those using the HOPS driver (Genesis MX) should be supported without modification. Other series using a different driver will require a new case to each case structure in the main class methods.
- **Laser Quantum:** all lasers in this series currently support a common RS232 protocol, so new lasers from this company should be supported without modification.
- **Thorlabs:** any “KCubeDCServo” type device is probably supported without modification. For other rotation stage devices, *Initialize.vi* and *Set Power.vi* may need to be modified.

Sync Device:

- Many National Instruments DAQ devices are likely supported by simply modifying the NI-DAQmx physical channel addresses in the configuration file. However, we do not recommend this path because the jitter of triggering from software is not very good.
- Other Arduino-type devices can be supported if they can be programmed to accept the same communication protocol as those already supported by FLASH.

6.2 CREATING A NEW *DEVICE* ACTOR

Below is the general process for creating a new *Device* actor, inheriting from an existing HAL interface class, such as *Camera*, *Laser*, *Sync Device*, etc.

1. Before you begin, create a stand-alone test VI to implement all planning functions of the target device driver. It is easier to troubleshoot errors and explore device quirks outside of the complexity of FLASH and the Actor Framework. If implemented as a series of helper VIs, these can be re-used in the new actor's methods. Be sure to fully test all planned workflows this way.
2. Create a new .lvlib in the "Hardware Devices" project virtual folder and save it in a new folder of the same name under the "Hardware" folder on disk.
3. Create a new class inheriting from the desired HAL class and save in the same folder as the lvlib.
4. Add the new lvclass to the "always included" list under the "source files" tab of the project build specification.
5. Add any device-specific data to the properties in the .ctl file associated with the class. Most often this matches the relevant part of the *Hardware Configuration* typedef.
6. Right-click on the lvclass and select "New->VI for Override..." and select a method to override from the HAL parent class. The list includes all parent classes, many of which are irrelevant. Some classes require the child to call the parent method, so it is best to look at other concrete classes of this device type to determine the expected pattern.
 - Override *Initialize.vi* (sometimes *Create Instance Impl.vi*) to connect to the device and perform any startup actions such as calibration.
 - Override *Shut Down Device.vi* (from *Device* class) with code to leave the device in a safe state, free up any resources, and disconnect.
 - Override *Handle Error.vi* (from Actor Framework) to deal with any errors that propagate to the output terminal of class methods. Unhandled errors will kill the actor.
 - Other override methods will be determined by the specific HAL class.
7. Append a new enum entry in the hardware configuration cluster for that device type. Generally located under "Resources/Type Defs". This will likely cascade further changes in the codebase. Check the Error List to see if any references need to be updated manually.
8. Update the relevant *Application/Init <HW>.vi* to include a case to include the file path for the new class. In some cases, such as the *Stage* class, it is the parent's *Initialize.vi* method that needs to be modified.
9. Run FLASH, selecting the new device in the hardware configuration, to test the new actor.

7 COMPILING AN EXECUTABLE

Except for testing and development, FLASH should always be run as a compiled binary. To compile the executable, follow these steps. Version numbers are specified as MAJOR.MINOR.BUGFIX.BUILD. Major versions break significant functionality users may depend on (such as migrating the new file formats). Minor versions implement significant new features. Bugfixes are slight changes to the code that only correct a specific bug and add no new features. The build number is used internally for development purposes and roughly corresponds to each commit in the source code repository.

- 1) In the project, double-click on “Build Specifications->FLASH”.
- 2) Click on the “Version Information” tab and update the version number. Build number should be set to be automatically incremented with each build.
- 3) If new files are required that LabVIEW may not find by traversing the VI dependencies (such as image and dll files or classes referenced only by file path), these can be added in the “Source Files” page.
- 4) Click “Build” to compile the project. This will often modify the .lvproj file.
- 5) The destination for the compiled project should be “C:\FLASH\testing” for unstable versions to be tested and “C:\FLASH\master” for stable releases deployed to production workstations.
- 6) Use “Resources\Post-Build Action.vi” to run any actions after the executable is built to prepare for deployment, such as removing temporary files. This script also moves .dll files to the appropriate place where the executable will search for them, which is often in the root folder with FLASH.exe for easy linking at run time.

8 C++ LIBRARIES

8.1 CONVERSION TO TIFF (DCIMAGE2TIFF)

As mentioned above, conversion from camera-specific intermediate files to TIFF is performed in a separate C++ library called `dcimg2tiff`. This library loads the individual movie files associated with distinct cameras (spectral bands), performs vertical/horizontal flips, and tiles them within one image for each frame. This approach produces images that mimic those produced with image-splitting optical devices such as the Cairn MultiCam.

For Hamamatsu cameras, the intermediate data is stored as `.dcimg` files, which are read using the `DCIMGAPI` library from Hamamatsu. The `DCIMG` format varies significantly between versions and should not be read directly.

For Photometrics, pixel data for each camera are saved to `.raw` files along with a “`ImageJ_import_CamN.txt`” file, where N is the camera index (0..4). This text file includes two critical numbers that can’t be derived directly from the raw files and are used by `dcimg2tiff` to read them. “Offset to first image” is the byte offset to the first pixel and “Gap between images” is the number of padding bytes between images. These extra bytes primarily serve to align the images to disk block boundaries for faster reading and writing. There is also a short header at the beginning of the `.raw` file, but its contents are undocumented and seem inconsistent between versions.

The conversion process is very disk intensive, so only one conversion happens at a time. Care should be taken in optimizing the code by introducing parallelized disk operations because these may starve the camera driver’s hard disk recorder from writing frame data to disk, which will result in dropped frames.

8.2 PARTICLE COUNTER

By default, FLASH uses a MATLAB node to count the number of particles (PSF maxima) in the field of view. The code is copied from SPARTAN. This creates a dependency that we ultimately want to remove. Towards this end, FLASH also includes a C++ module using OpenCV to perform these calculations. The algorithms are not identical but perform similarly. This module can introduce long application freezes that seem to be associated with context switches between LabView threads, which force OpenCV to reload static resources. Since no solution is obvious, this module is currently unused.

9 TEAM DEVELOPMENT AND OTHER TIPS

Due to LabVIEW's graphical nature and binary source code, version control has some unique challenges but is doable. Below are some tips to make the process as painless as possible:

- 1) All developers on the project must use the exact same version of LabView.
- 2) When moving files, always do so in LabView using the "Move on disk" option in the Files view – do not move files outside of LabView! The source files (and possibly others) will be modified in this process because LabView maintains file paths for all references.
- 3) It is best to use a single shared computer for producing compiled versions. This ensures the build number is always correct and avoids linking issues where the absolute path to some resources is slightly different.
- 4) Only commit files that were intentionally changed or cascaded from a known dependency (such as an updated typedef). LabVIEW tends to alter files that the user did not touch for obscure reasons. In particular, take care when committing the main project file ("lvproj") to avoid conflicts with other branches/developers.
- 5) Never commit the ".aliases" and ".lvpls" files, which are workstation specific. Use .gitignore if possible to exclude them explicitly.
- 6) Although possible, merging multiple commits to a single file isn't recommended. If multiple contributors are working on the code base at the same time, take extra effort to minimize the scope of changes to avoid possible conflicts. If larger scale changes are needed (such as widely-used typedefs), communication is essential to coordinate commits without conflicts.
- 7) It is possible to diff or merge commits using LVDIFF and LVMERGE utilities, but this requires some work to set up and is finicky.
https://labviewwiki.org/wiki/Set_up_differencing_capabilities
- 8) Changes to typedefs (including enums) will cascade through every file that references it, which increases the likelihood of merge conflicts. Avoid changes to typedefs when possible. Typedefs are only needed when it is desirable for every piece of code using that typedef to need review if the data structure is modified. When used, keep the scope of any typedefs confined only to where it is needed. In some cases, it is appropriate to pass such data as a Variant type to avoid dependencies, such as when the handling class/method doesn't inspect the message contents.
- 9) Review this article about separating source and compiled code. If this option is not selected, source files are more likely to be altered indirectly and cause issues for version control.
<https://www.ni.com/docs/en-US/bundle/labview/page/facilitating-source-control-by-separating-compiled-code-from-vis-and-other-file-types.html>

10 TROUBLESHOOTING

This section provides tips for troubleshooting errors and program crashes.

10.1 ERROR CODES

Below is a list of standard error codes for FLASH.

Code	Actor	Error description
5981	Stage	Soft limit exceeded
5982	Stage	Invalid axis
5989	Stage	Other error
5972	Microscope	Ti2 stand not detected
5979	Microscope	Other error
5961	Sync	Arduino configuration error
5962	Sync	Arduino acquisition period too short
5969	Sync	Other error
5951	Power Meter	Initialization failed (device not detected)
5941	Camera	Initialization error
5942	Camera	Invalid acquisition settings
5943	Camera	Disk streaming failed
5944	Camera	Not enough disk space
5949	Camera	Other error
5931	Autosampler	Initialization error
5932	Autosampler	Invalid response
5933	Autosampler	Protocols error
5938	Autosampler	Wait for Idle Timeout
5939	Autosampler	Other error
5921	Tiff Writer	Operation cancelled
5921	Tiff Writer	Invalid input file(s)
5922	Tiff Writer	Invalid output file(s)
5923	Tiff Writer	Potentially dropped frame (timestamp jump)
5929	Tiff Writer	Other error
5910	Any	Not implemented
5911	Any	Operation timed out
5912	Any	Operation failed

10.2 LOG FILES

FLASH logs the execution of Application actor-level functions, along with a timestamp. This information can be retrieved using *Resources/Log Message.vi*, which is a functional global variable containing the log information. The log data is also displayed in the Main Window's "log" tab as well as saved to disk in real time at "C:\temp\FLASH.log", which is particularly useful in the case of program crashes.

```
17:20:53.96 -- Set filter=1, path=2
```

DCAM, if used, logs commands, return values, and any error codes in C:\temp\tmllog.txt.

```
1.944 TMCC_GETAREA_40 -> i=0
```

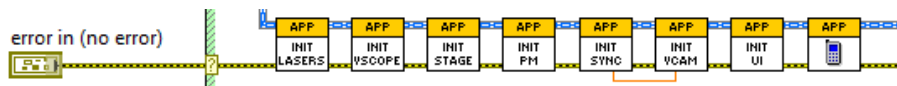
```
1.953 TMCC_GETAREA_40 OK hov=0 vov=0 hwv=2048 vwv=2048
```

dcimg2tiff logs any errors in C:\temp\dcimg2tiff.log.

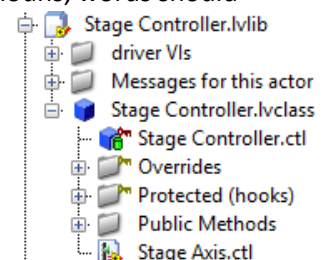
11 STYLE GUIDE

This section describes programming style to help keep the code base consistent and “readable”.

1. **Wire diagrams should fit within one screen** to avoid scrolling. To achieve this, avoid unnecessary white space, keep the scope of each VI to one well-defined purpose, and extract code blocks to new helper VI’s where it makes sense. This also helps to minimize chances of merge conflicts in team development.
2. **Terminals should use a consistent 4x2x4 pattern**, with inputs on the left and outputs on the right. Avoid wiring in more than 4 inputs – use clusters instead.
 - a. Top left terminal is the primary input and top right is primary output. This is usually an object wire (for class methods) or resource handle (for file access, etc.).
 - b. The bottom left should be error in and bottom right error out.
 - c. Middle left terminals are for input parameters.
 - d. Middle right terminals are outputs, such as results of a method call.
 - e. Center terminals are often used for enum inputs because this allows long names to above or below the VI node in the block diagram to save space.
 - f. Center terminals should not be used as outputs.
3. **Align VI nodes from left to right** so that the error wire is straight from input to output. This helps keep program flow clear and linear. Use the error wire as the primary means of controlling parallelization within one VI.



4. **Keep icons simple**, with a top bar to identify the module/class and the middle part giving a brief text description of the function. Avoid custom icon images. An exception are HAL override methods (“hooks”), which include a downward blue arrow.
5. **Comments** on wire diagrams should describe obscure constant values, algorithms, and document assumptions.
6. **VI descriptions** should give a one sentence summary of the purpose of the function and describe each input and output terminal.
7. **Naming:** method names should be verbs, class member names should be nouns, words should be separated by spaces, and each word should be capitalized.
8. **Use virtual folders** to organize class methods and set access scope explicitly for each virtual folder. See an example at right. Outside of classes, use auto-populating virtual folders so that the project and disk organization matches.



9. Helper VIs, typedefs, and other resources should be owned by the lplib that uses them and located in the same folder on disk. Use the Resources folder only sharing across modules.
10. Use the built-in Actor Framework “Create Message” and “Rescript this message” macros to create and update messages. Do not manually create/modify them.